Java Programming Exercises

Multithreading and Concurrency

Exercise 1 — Producer/Consumer: Random Number

Build a Java program that uses two threads to produce and consume a sequence of random numbers, synchronized so each number is consumed exactly once.

`NumberProducer` thread

- In a loop: sleep 5 seconds (`Thread.sleep(5000)`), generate a random `int` from 1 to 10 inclusive, store it in a field shared with the consumer, and print it, e.g. `Produced: 7`.
- Keep looping until the generated number is 10 — that value is still handed off and printed like any other, then the thread exits the loop.

`WordConsumer` thread

- Waits for each new number to become available, converts it to its English word using a fixed lookup table: `{1:"one", 2:"two", 3:"three", 4:"four", 5:"five", 6:"six", 7:"seven", 8:"eight", 9:"nine", 10:"ten"}`, and prints it, e.g. `Consumed: seven`.
- After consuming 10 ("ten"), the thread exits.

Synchronization

Use a shared lock object with a boolean `ready` flag: the producer sets the number, sets `ready = true`, and calls `notify()` inside a synchronized block; the consumer loops `while (!ready) { lock.wait(); }`, consumes the number, sets `ready = false`, then loops back to wait for the next one. This guarantees the consumer never reads the same number twice and never reads before it's produced.

`main()`

Start both threads, then call `join()` on both so the program doesn't exit until the sequence (ending on 10) has fully completed. The repeating behavior lives inside the producer's `run()` method — `main()` itself doesn't need its own loop.

Exercise 2 — Sum of an Array Using a Thread

Build a Java program that uses a thread to compute the sum of an integer array.

`SumThread` class

- Fields: `int[] data` (set via the constructor), `int result` (populated by `run()`).
- `run()` iterates over `data` and stores the total in `result`.
- Example array: `int[] numbers = {4, 8, 15, 16, 23, 42};`

Main class

- Create the thread with the array above, call `start()`, then call `join()` before reading `result` — reading it without joining first is a race condition, since the sum might not have been computed yet.
- Print the final sum, e.g. `Sum: 108`.

Exercise 3 — Printing Numbers in Random Order Using a Thread

Build a Java program that uses a thread to print the integers 1 through 10 in a random order.

`RandomOrderThread` class

Build a `List<Integer>` containing 1 through 10, call `Collections.shuffle()` on it once, then in `run()` print each element on its own line.

Main class

Start the thread and `join()` it before the program exits.

Exercise 4 — Countdown Timer Using a Thread

Build a Java program that uses a thread to count down from 10 to 1 and announce liftoff.

`CountdownThread` class

In `run()`, loop from 10 down to 1, printing each number, with `Thread.sleep(1000)` between each so the countdown takes roughly 10 seconds. After printing 1, print `Liftoff!` and exit the loop.

Main class

Start the thread and `join()` it before the program exits.

Exercise 5 — Queue Operations Using a Thread

Build a Java program that uses a thread to add to and remove from a queue, printing its contents after each change.

`QueueThread` class

- Use a `Queue<Integer>` (e.g. `new LinkedList<Integer>()`) as an instance field.
- In `run()`, add a fixed sequence of values — e.g. 10, 20, 30, 40 — to the queue one at a time, printing the queue's contents after each addition.
- Then remove elements one at a time with `poll()`, printing the removed value and the queue's remaining contents after each removal.

Main class

Start the thread and `join()` it before the program exits.

Exercise 6 — Elapsed Time Counter Using a Thread

Build a Java program that uses a thread to print the elapsed time once per second for ten seconds.

`TimerThread` class

- Record the start time with `System.currentTimeMillis()` before the loop begins.
- In `run()`, loop 10 times: sleep 1 second, then print the elapsed time in seconds since the start, e.g. `Elapsed: 3s`.
- After the 10th print, the thread exits.

Main class

Start the thread and `join()` it before the program exits.

Exercise 7 — Two Threads Modifying a Shared Array

Build a Java program where one thread holds a shared array and a second thread modifies it.

`ArrayHolderThread` — array holder

Holds the shared field `int[] numList = {1, 3, 4, 6, 7};` and exposes it (e.g. via a getter) to `ArrayModifierThread`. Its `run()` method can simply print a message confirming the array is ready.

`ArrayModifierThread` — array modifier

In `run()`, adds 10 to every element of the shared array, accessed via a reference passed in through `ArrayModifierThread`'s constructor.

Main class

Start `ArrayHolderThread` and call its `join()` so the array is guaranteed ready, then start `ArrayModifierThread` and call its `join()`, then print the final array contents with `Arrays.toString()`. Expected output: `[11, 13, 14, 16, 17]`.

Exercise 8 — Odd/Even Numbers Printed in Order by Two Threads

Build a Java program where two threads take turns printing even and odd numbers in increasing order.

Setup

Read a positive integer n from the keyboard before starting either thread.

`EvenThread`

Prints even numbers starting from 0: 0, 2, 4, ... up to and including n if n is even, or up to n − 1 if n is odd.

`OddThread`

Prints odd numbers starting from 1: 1, 3, 5, ... up to and including n if n is odd, or up to n − 1 if n is even.

Coordinating the output

- Use a shared lock object and a boolean `evensTurn = true` flag with `wait()`/`notifyAll()`: `EvenThread` prints while `evensTurn` is true, then flips it to false and notifies; `OddThread` does the mirror image while it's false. This is what produces the exact 0, 1, 2, 3, 4, 5, 6, 7, 8 ordering shown in the example below.
- Print each number as `"Even thread: " + n` or `"Odd thread: " + n` to match the example output.
- After both threads finish (join them from the main thread), print `"Done"`.

Example

For n = 8:

```
Even thread: 0
Odd thread: 1
Even thread: 2
Odd thread: 3
Even thread: 4
Odd thread: 5
Even thread: 6
Odd thread: 7
Even thread: 8
Done
```

Exercise 9 — Alternating Uppercase/Lowercase Letters by Two Threads

Build a Java program where two threads take turns printing uppercase and lowercase letters in alphabetical order.

`UppercaseThread`

Prints the letters 'A' through 'Z', one per turn.

`LowercaseThread`

Prints the letters 'a' through 'z', one per turn.

Coordinating the output

Use a shared lock object with a boolean flag and `wait()`/`notifyAll()` turn-taking: `UppercaseThread` prints a letter, flips the flag, and notifies; `LowercaseThread` does the mirror image — so the sequence is A, a, B, b, C, c, ... Z, z. After both threads finish all 26 letters, print `"Done"`.

Exercise 10 — Day-of-the-Week Producer/Consumer (Two Languages)

Build a Java program where one thread produces a random day of the week in Vietnamese and a second thread reports its English equivalent.

Day list

Two index-aligned arrays:

- English: Monday, Tuesday, Wednesday, Thursday, Friday, Saturday, Sunday
- Vietnamese: Thu hai, Thu ba, Thu tu, Thu nam, Thu sau, Thu bay, Chu nhat

`DayProducer` thread

After a 1-second delay, picks a random index from 0–6, prints the Vietnamese name at that index, and hands the index to the consumer thread.

`DayConsumer` thread

Waits for the index from the producer, then prints the English name at the same index.

Synchronization

Shared lock object + boolean `ready` flag with `wait()`/`notify()`: producer sets the value and notifies; consumer waits until it's ready.

`main()`

Start both threads, join both, then exit — this is a single one-shot demonstration (produce one day, consume it once), not a repeating loop.

Exercise 11 — Bank Deposit/Withdrawal Threads

Build a Java program where two threads deposit into and withdraw from a shared bank balance, safely coordinating access to it.

Shared account

A single shared field `int balance = 0;`, updated only inside synchronized methods (or blocks synchronized on the same shared lock object) so a deposit and a withdrawal can never interleave mid-update.

`DepositThread`

Runs 5 deposit cycles. Each cycle: sleep 3 seconds, deposit a random amount between 100 and 1000 inclusive, and print, e.g., `Thread 1 deposits 500, total balance is 500.`

`WithdrawThread`

Runs 5 withdrawal cycles. Each cycle: sleep 4 seconds, generate a random amount between 100 and 500 inclusive, then — inside the same synchronized method/lock used for deposits — check whether `balance >= amount`; if so, subtract it and print, e.g., `Thread 2 tries to withdraw 300, total balance is 200.` If not, print an error instead and leave the balance unchanged, e.g., `Thread 2 tries to withdraw 800, insufficient balance (200 available).`

`main()`

Start both threads, `join()` both, then print the final balance.

Exercise 12 — Word Uppercase Producer/Consumer

Build a Java program where one thread converts words to uppercase and hands each result to a second thread for display.

Shared data

```
String[] words = {"Sai Gon", "Da Nang", "Hue"};
```

`UppercaseThread`

Iterates `words` in order; for each one, converts it to uppercase with `.toUpperCase()` and hands the result to the display thread.

`DisplayThread`

Waits for each converted word and prints it, e.g. `SAI GON`.

Synchronization

Use a shared lock object with `wait()`/`notify()` so `DisplayThread` never prints the same word twice and never prints before the next one is ready.

Main class

Start both threads and join them; after all 3 words are processed, both threads exit.

Exercise 13 — String Reversal Producer/Consumer

Build a Java program where one thread reverses strings and hands each result to a second thread for display.

Shared data

```
String[] data = {"Hello", "Java", "Programming"};
```

`ReverseThread`

Iterates `data` in order; for each string, reverses it (e.g. via `new StringBuilder(s).reverse().toString()`) and hands the result to the display thread.

`DisplayThread`

Waits for each reversed string and prints it.

Synchronization

Use `synchronized`, `wait()`, and `notify()` on a shared lock to hand off each value in turn.

Main class

Start both threads and join them.

Exercise 14 — Square-Number Producer/Consumer

Build a Java program where one thread squares integers and hands each result to a second thread for display.

Shared data

```
int[] numbers = {10, 25, 36, 49, 64};
```

`SquareThread`

Iterates `numbers` in order; for each integer, computes its square and hands the result to the display thread.

`DisplayThread`

Waits for and prints each squared value.

Synchronization

Use `synchronized`, `wait()`, and `notify()` on a shared lock to hand off each value in turn.

Main class

Start both threads and join them.

Exercise 15 — Book Genre Counter (Multithreaded)

Build a Java program where two threads concurrently update and read a shared map of book counts by genre.

Shared data

`Map<String, Integer> genreCounts`, seeded with e.g. `Novel: 10, Science: 7, Literature: 12`. Every read or update must happen inside a synchronized block on the map (or use a `ConcurrentHashMap`) so the two threads never corrupt each other's updates.

`GenreUpdateThread`

Runs 10 cycles. Each cycle: sleep 1 second, pick a random genre key, increment its count by 1 (as a synchronized read-modify-write), then print the genre and its new count.

`TotalTrackerThread`

Runs 10 cycles. Each cycle: sleep 1 second, compute the current total across all genres (sum of the map's values, read under the same lock used above), and print the running total.

`main()`

Start both threads, `join()` both, then print the final map contents.

Exercise 16 — Product Stock In/Out Counter (Multithreaded)

Build a Java program where two threads concurrently update a shared map tracking received and shipped quantities per product.

Shared data

`Map<String, int[]> stock`, where each value is a 2-element array `[received, shipped]`, seeded with e.g. `Phone: {20, 5}, TV: {15, 3}`. Keeping received/shipped together in one array (rather than two separate maps) means both fields update under the same lock and can't drift out of sync.

`ReceivingThread`

Runs 10 cycles. Each cycle: sleep 1 second, pick a random product, synchronized-increment its received count by 1, then print the product and its updated received quantity.

`ShippingThread`

Runs 10 cycles. Each cycle: sleep 1 second, pick a random product independently of the receiving thread, synchronized-decrement its shipped count by 1, then print the product and its updated shipped quantity. Don't let a shipped count go below 0 — if the decrement would, skip that cycle and print a message noting it instead.

`main()`

Start both threads, `join()` both, then print the final map contents.

Exercise 17 — Hotel Room Booking Counter (Multithreaded)

Build a Java program where two threads concurrently update a shared map tracking booked and returned rooms per hotel.

Shared data

`Map<String, int[]> rooms`, where each value is a 2-element array `[booked, returned]`, seeded with e.g. `Hotel A: {20, 5}`, `Hotel B: {15, 3}`.

`BookingThread`

Runs 10 cycles. Each cycle: sleep 1 second, pick a random hotel, synchronized-increment its booked-room count by 1, then print the hotel name and the new booked-room count.

`ReturnThread`

Runs 10 cycles. Each cycle: sleep 1 second, pick a random hotel independently of the booking thread, synchronized-decrement its returned-room count by 1, then print the hotel name and the new count. Don't let a returned count go below 0 — if the decrement would, skip that cycle and print a message noting it instead.

`main()`

Start both threads, `join()` both, then print the final map contents.

Exercise 18 — Three-Thread Day-Name Pipeline

Build a Java program where three threads form a pipeline that generates a day name, normalizes and translates it, then logs the result to a file.

Day mapping (English → Vietnamese)

Monday → Thu Hai, Tuesday → Thu Ba, Wednesday → Thu Tu, Thursday → Thu Nam, Friday → Thu Sau, Saturday → Thu Bay, Sunday → Chu Nhat.

`DayGeneratorThread` (Thread 1)

Runs 5 cycles. Each cycle: sleep 2 seconds, pick a random day name from the English list, then randomly convert the whole string to either `toUpperCase()` or `toLowerCase()`, and hand the mixed-case string to Thread 2.

`NormalizerThread` (Thread 2)

Waits for each string from Thread 1, normalizes its capitalization (first letter upper, rest lower — e.g. `s.substring(0,1).toUpperCase() + s.substring(1).toLowerCase()`), looks up the Vietnamese equivalent from the mapping table above, displays it, and hands both the normalized English name and the Vietnamese name to Thread 3.

`LoggerThread` (Thread 3)

Waits for each pair from Thread 2 and appends one line to `days.log` in the format `English - Vietnamese` (e.g. `Monday - Thu Hai`), using a `BufferedWriter`/`FileWriter` opened once at the start and closed after all 5 cycles are logged. Example file contents after a run:

```
Monday - Thu Hai
Sunday - Chu Nhat
```

Synchronization

Use two separate shared lock objects, each with its own handoff field and `wait()`/`notify()` pair — one between Thread 1 and Thread 2, another between Thread 2 and Thread 3 — so each stage only processes a value once it's actually ready.

`main()`

Start all three threads, `join()` all three, then print a confirmation, e.g. `5 entries written to days.log`.

Exercise 19 — Fragrance Name Generator and Translator (Two Threads)

Build a Java program where one thread generates a fragrance name in French and a second thread translates and displays it in English, repeated three times.

Translation table

Lavande → Lavender, Rose → Rose, Vanille → Vanilla, Citron → Lemon, Jasmin → Jasmine.

`FragranceGenerator` class extends `Thread`

After `Thread.sleep(1000)`, picks a random fragrance name from the French list above and prints it, e.g. `Generated: Rose`.

`FragranceTranslator` class extends `Thread`

Waits until the generator has produced and displayed a fragrance, then looks up and prints its English translation, e.g. `Translated: Rose`.

Synchronization

Shared lock object with `wait()`/`notify()` so the translator never reads before the generator has written its value.

`main()`

Run the generate-then-translate cycle 3 times total. Each round: create a new pair of `FragranceGenerator`/`FragranceTranslator` threads (or reuse the same synchronization object across 3 sequential rounds), start them, and `join()` both before starting the next round — this guarantees the 3 rounds print in order and never overlap.

Exercise 20 — Freelance Skill and Project Description Matcher (Two Threads)

Build a Java program where one thread picks a random skill from a map and a second thread displays its matching project description.

Shared data

`Map<String, String> skillToProject`, seeded with e.g. `Java: "Develop backend systems"`, `Python: "AI model training"`.

`SkillGeneratorThread`

After a 1-second delay, picks one random key from the map and prints it, e.g. `Skill: Java`.

`ProjectDisplayThread`

Waits for the generated skill, looks up its value in the map, and prints it, e.g. `Project: Develop backend systems`.

Synchronization

Shared lock object with `wait()`/`notify()`.

`main()`

Single one-shot demonstration: start both threads, join both, then exit.

Exercise 21 — Getting Familiar with Synchronizing Two or More Threads

Build a Java program where one thread generates a random number and a second thread reports whether it's even or odd.

`NumberGeneratorThread`

Generates one random `int` from 0 to 50 inclusive and hands it to the second thread.

`ParityThread`

Waits for the number, then prints whether it's even or odd, e.g. `42 is even`.

Synchronization

Shared lock object with a boolean `ready` flag and `wait()`/`notify()`, ensuring `ParityThread` only runs after `NumberGeneratorThread` has produced its value.

`main()`

Start both threads, join both.

Exercise 22 — Normalizing an Array of Names Using Two Threads

Build a Java program where one thread displays raw names one at a time and a second thread normalizes their capitalization.

Shared data

```
String[] listName = {"nGuYeN hOa bInH", "lE tHaNh hAi", "tRan mAnH tIeN", "lE qUAnG qUaN"};
```

`NameDisplayThread`

Every 1 second, prints the next raw (unnormalized) name from the array in order, then hands it to the second thread.

`NormalizeThread`

Waits for each name, normalizes it to proper capitalization — every word's first letter uppercase, the rest lowercase, e.g. `nGuYeN hOa bInH → Nguyen Hoa Binh` — and prints the normalized result.

Synchronization

Shared lock object with `wait()`/`notify()` handoff.

`main()`

Start both threads, join both; after all 4 names are processed, both threads exit.

Exercise 23 — Multithreaded Student Record Management

Build a Java program where one thread reads student records from the keyboard and two more threads independently write them to a text file and to object files.

`Student` class

- Fields: `String studName`, `int studAge`, `String address`.
- A constructor setting all three fields; the name is converted to uppercase when stored, regardless of how it was entered — age and address are kept exactly as entered.

- Override `toString()` to return the three fields space-separated, matching the `Sinhvien.txt` example format below, e.g. `NGUYEN VAN A 19 Ha noi`.
- `implements Serializable` — required for the object-file writer below.

Coordination design

- Use two `BlockingQueue<Student>` instances (e.g. `LinkedBlockingQueue`), `queueForText` and `queueForObject`. `InputThread` puts each newly entered student onto both queues; `TextWriterThread` only consumes from `queueForText`; `ObjectWriterThread` only consumes from `queueForObject`. This lets the two writer threads work independently instead of contending over one shared value.
- Use a sentinel `Student` object (e.g. one whose `studName` is the reserved marker `"__END__"`) to signal both queues that input has finished.

`InputThread` (Thread 1)

Loops: prompt for and read `studName`, `studAge`, and `address` from the keyboard; construct a `Student`; print it via `toString()`; put it onto both queues; then ask "Enter another student? (y/n)". On 'n', put the sentinel value onto both queues (so the two writer threads know to stop) and end the input loop.

`TextWriterThread` (Thread 2)

Loops: take the next `Student` from `queueForText`; if it's the sentinel, close the writer and exit; otherwise append one line to `Sinhvien.txt` (created if it doesn't exist) in the format shown below, flushing after every write so data isn't lost if the program is interrupted. Example file contents after a few entries:

```
NGUYEN VAN A 19 Ha noi
NGUYEN THI B 22 Hai phong
TRAN VAN C 33 HCM
LE TRUNG D 24 Da Nang
```

`ObjectWriterThread` (Thread 3)

Loops, keeping a running counter starting at 1: take the next `Student` from `queueForObject`; if it's the sentinel, exit; otherwise serialize it with `ObjectOutputStream` to a new file named `Sinhvien_01.dat`, `Sinhvien_02.dat`, etc. (counter zero-padded to 2 digits), then increment the counter.

`main()`

Start `TextWriterThread` and `ObjectWriterThread` first (so they're ready to consume), then start `InputThread`, then `join()` all three threads before the program exits.